LagPilot: Learning When to Scale Kafka Consumers with Q-learning

CSCI3235 Reinforcement Learning, Midterm I

Aliyev Nurislam, Student ID: 24B030024, 26 October 2026

1. Problem and RL suitability

The task. An online shop sends every order to a Kafka topic called `orders`. The topic has 4 partitions. A group of workers, called *consumers*, reads and processes the orders. The decision-maker is an autoscaler. Every 10 seconds it chooses one action: keep the number of consumers, add one, or remove one (from 1 to 4 consumers). Traffic is quiet at night (20 orders per second, RPS) and busy at midday (120 RPS). Sometimes a flash sale doubles the traffic. If there are too few consumers, the queue of waiting orders (the *lag*) grows, and orders wait longer than our promise of 30 seconds (the *SLA*). If there are too many consumers, we pay for idle servers. The hard part: every change in the number of consumers starts a *rebalance*. During a rebalance the whole group stops working for 20 seconds. Our goal is short waiting times, few SLA violations and few servers, without changing the consumer count too often.

Why RL fits. The decisions are connected in time. Orders that are not processed now are still waiting in the next step. A rebalance started now reduces processing in the next steps. A consumer added before a flash sale helps later. The agent does not know the traffic in advance, so it must learn from experience when a change is worth its cost.

Non-RL alternative. The industry rule KEDA sets `consumers = ceil(lag / lagThreshold)`. Another option is capacity planning with Little's Law. These simple methods are better when traffic is stable, when rebalances are cheap, or when the team needs a rule that is easy to explain.

Capacity check. One consumer processes 50 orders per second. At the peak, 120 / 50 = 2.4, so we need 3 consumers. At night, 1 consumer is enough. A flash sale at the peak brings 240 RPS, but 4 consumers can process only 200. So the lag must grow during a big sale. No policy can stop this; a good policy can only manage it. **Link to my work:** I am a backend developer. My *Backend for Highloaded Environment* course covers Kafka, capacity planning, SLAs and autoscaling.

2. MDP formulation

Episode. One step is 10 seconds. One episode is 120 steps, a short "day". The task is **episodic**. At the start there are 2 consumers and no waiting orders.

Actions. `keep`, `scale_up` (+1, max 4), `scale_down` (−1, min 1). If an action is not possible, or a rebalance is running, it counts as `keep`. More than 4 consumers is useless, because each partition goes to only one consumer.

Transitions (random). A change starts a rebalance of 2 steps; the new count starts when it ends. The number of new orders in a step is random (Poisson, with mean RPS × 10). A flash sale (traffic × 2) starts with a 4% chance per step and lasts 5–15 steps. Processed orders = min(lag + new orders, consumers × 50 × 10), or 0 during a rebalance. New lag = lag + new orders − processed. The waiting time comes from Little's Law: `delay = lag / (consumers × 50)`.

State (288 states). The agent sees five things, each put into a few groups: delay (under 5 s, under 15 s, under 30 s, 30 s or more), lag trend (falling, stable, rising), traffic level (under 40, under 90, 90+ RPS), number of consumers (1–4), and "is a rebalance running" (yes/no). 4 × 3 × 3 × 4 × 2 = 288.

Reward. The reward is minus the cost of the step: $r = -\left(\min(\text{delay}, 120)/10 + 2 \times \text{consumers} + 5 \times [\text{delay} > 30]\right)$. We pay for waiting orders, for servers, and extra for breaking the SLA. There is no direct cost for a rebalance. Its cost comes later, through the growing lag. The agent must discover this by itself.

Discount. γ = 0.95. This means the agent looks about 20 steps (200 seconds) ahead. That is enough to "see" a 20-second rebalance and a typical flash sale. At the last step there is no future, so the target is just the reward. The environment is random. The training policy is random too (ε-greedy). The final policy is fixed (always the best action).

Markov property. The state is only partly Markov. The agent does not see the exact lag, the exact traffic, whether a flash sale is running, the time of day, or how long the rebalance will last. Section 5.4 tests what happens when we remove the traffic level.

One real step (from `results/transition_example.json`). State: low delay, stable lag, low traffic, 2 consumers, no rebalance. Action: **scale_down**. Result: 196 new orders arrive, 0 are processed because the rebalance has started, so the lag is 196 and the delay is 196 / (2 × 50) = 1.96 s. Reward = −(0.196 + 4 + 0) = **−4.196**. Next state: low delay, rising lag, low traffic, 2 consumers, rebalance running. The Q-values in the first state were keep −81.71, scale_up −79.37, scale_down −76.63, so the agent chose scale_down. At night one consumer is enough.

3. Method and update rule (tabular Q-learning)

The agent keeps a table Q(s, a) with 288 rows and 3 columns. It starts with zeros. Each value is a guess: "how much total (discounted) reward will I get if I take action a in state s and then act well?" (Sutton & Barto, §6.5). After every step the agent updates one value:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

Q(s, a) is the old guess. **r** is the reward we just got. **γ max Q(s′, ·)** is the value of the best action in the next state, a bit reduced by γ. Together, **r + γ max Q(s′, ·)** is the *target*, a better guess based on one real step. **Target − old guess** is the error. **α = 0.1** means we move the old guess 10% toward the target. Because the target uses the *best* next action, Q-learning is *off-policy*: it learns the best policy while it still explores.

Using the table. During training the agent uses ε-greedy: with probability ε it tries a random action, otherwise it takes the action with the highest Q-value. ε goes down from 1.0 to 0.05 during the first 70% of training. During testing the agent always takes the best action and does not learn.

Baselines. KEDA rule: `clip(ceil(lag / 500), 1, 4)`, moving one step toward it. 500 was the best threshold for KEDA. **Static max:** always 4 consumers. **Random:** random actions.

4. Experimental setup

We train 10 agents with different random seeds (0–9), 3000 episodes each. Every 50 episodes we test the agent on 10 fixed days (seeds 2000–2009) to draw the learning curve. At the end, every agent and every baseline is tested on 50 new days (seeds 1000–1049) that were never used in training. We report the mean and the standard deviation. The simulator settings were chosen once, using only the baselines, and then frozen. 3000 episodes is enough, because the curve stops improving after ε reaches 0.05 (Figure 1). One command, `python -m lagpilot.experiments`, repeats everything in about 2 minutes. All numbers come from `results/*.csv`.

5. Results

5.1 Training curve

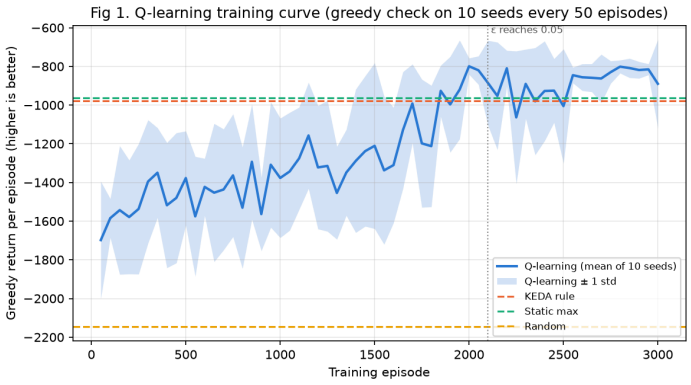


Figure 1: Greedy test return during training, mean ± std over 10 seeds. Dashed lines: baselines.

The score improves from about −1698 (episode 50) to about −800 (episode 2000). It ends above both KEDA and static max. But it is not fully stable: at some points the mean drops (−1005.2 at episode 2500) and the spread grows (std 228.2 at episode 3000). This happens because some agents change their policy late in training.

5.2 Final comparison

Policy	Return (higher is better)	Delay (s)	SLA broken (%)	Consumers	Rebalances
Q-learning	**−897.4 ± 264.2**	30.1 ± 84.0	14.7 ± 14.6	**2.68**	10.5
KEDA rule	−1002.6 ± 84.9	15.3 ± 2.3	10.5 ± 7.1	3.15	26.6
Static max	−961.5 ± 8.7	**0.8**	**0.0**	3.97	2.0
Random	−2126.7 ± 129.2	332.9	78.3	2.51	40.7

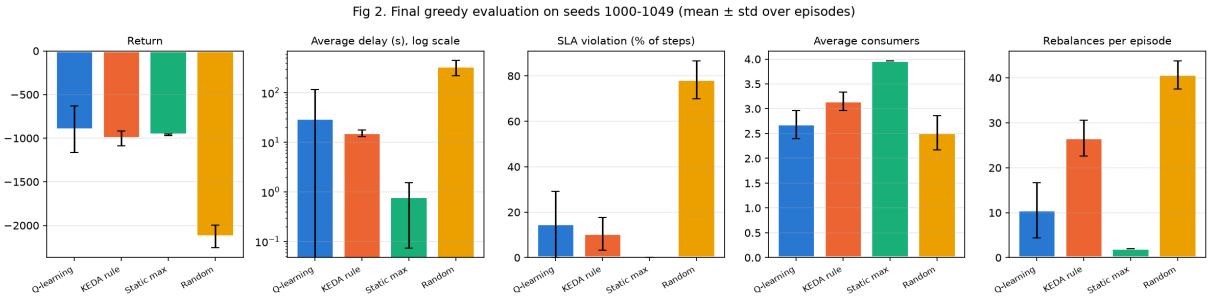


Figure 2: Final test on 50 new days (mean ± std over episodes).

Q-learning has the best average score. It uses fewer servers than KEDA (2.68 vs 3.15) and makes 2.5 times fewer rebalances. But it breaks the SLA more often (14.7% vs 10.5%). It accepts some waiting to save servers, because the reward allows this. The big ± value comes from **one failed agent**: 9 agents score between −874.8 and −772.7, but seed 2 scores −1537.8. The median is −829.7. On one test day (`fig4_episode_trace.png`), KEDA keeps removing and adding consumers (21 rebalances), while Q-learning keeps 4 consumers during the busy hours (14 rebalances).

5.3 Parameter study: how long a rebalance takes

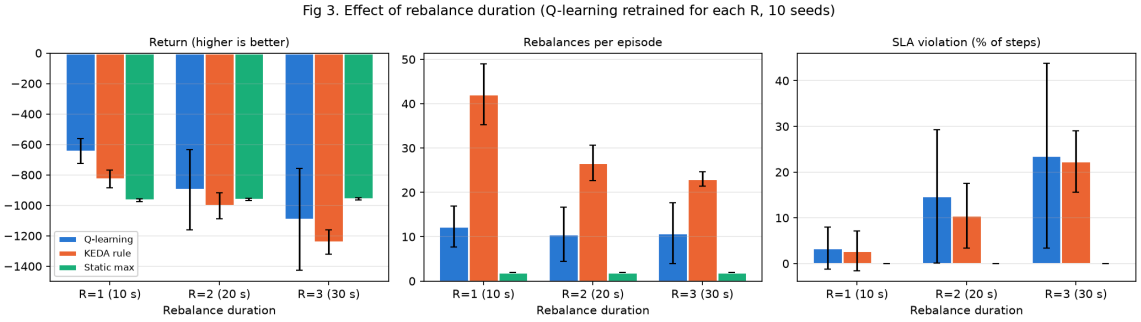


Figure 3: Q-learning retrained for each rebalance time (10 seeds), compared with KEDA and static max.

Rebalance time	Q-learning	KEDA	Static max	Q rebalances	KEDA rebalances	Q agents better than KEDA / static
10 s	−643.4 ± 82.3	−826.9	−966.5	12.3	42.1	10 / 10
20 s	−897.4 ± 264.2	−1002.6	−961.5	10.5	26.6	9 / 9
30 s	−1092.7 ± 333.4	−1239.7	−957.3	10.8	23.1	8 / 4

KEDA gets much worse when rebalances are slower, because it changes the consumer count too often. Q-learning beats KEDA at every rebalance time. But we expected the agent to scale less when rebalances are expensive, and this **did not happen**: it makes about 11 rebalances every time. At 30 s, **static max is the best policy on average**, and only 4 of 10 agents beat it.

5.4 Does the traffic level help? (Markov study)

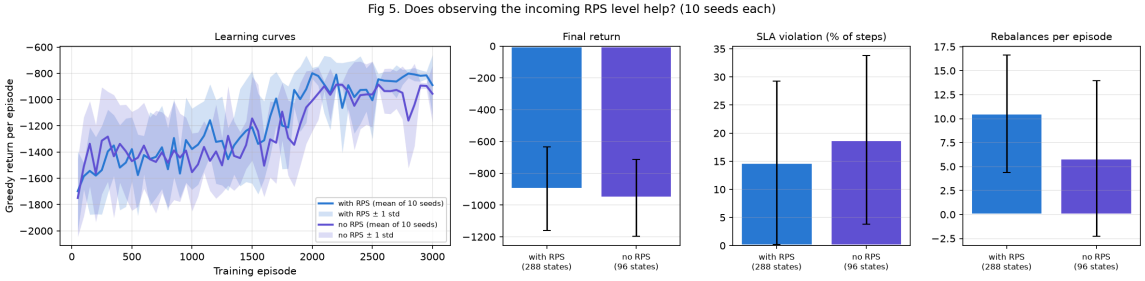


Figure 4: With traffic level (288 states) vs without it (96 states), 10 seeds each.

Without the traffic level, the score is −953.5 ± 241.8 (median −876.1). With it, the score is −897.4 ± 264.2 (median −829.7). SLA violations grow from 14.7% to 18.8% without it. The agent then sees only the *result* of traffic (the delay), not its *cause*, so it reacts later. Each version has one failed agent, so the medians show the difference more clearly.

6. Discussion, limitations, and next step

What failed. Seed 2 learned a bad policy (−1537.8). The policy map (`fig6_policy_heatmap.png`) also shows strange actions in rare states, for example scale_down when the delay is critical. These states were seen only a few times, so their Q-values are noisy. **Limitations.** The state groups are rough (31 s and 300 s look the same). There is no time-of-day information. We model the old "stop-the-world" rebalance; modern Kafka has a cheaper cooperative rebalance (KIP-429). Our KEDA rule has no cooldown, which helps Q-learning. The simulator is simpler than real life. **Next steps.** A smaller or decreasing α, choosing the best Q-table on test days, a time-of-day feature, SARSA, KEDA with a cooldown, and tests on real Kafka (the `demo/` folder is a first step but is not part of the results).

7. AI Use Statement

Tool: Claude Code (Anthropic, Claude Opus 5.5). **Used for:** writing the first versions of the simulator, agents, experiments and figures from my plan; suggesting the one-time settings tuning (baselines only); drafting the README and this report from the CSV results; the optional demo. **What I verified myself:** I ran the live Kafka demo myself (API, load generator and agent) and checked in the agent log that the trained policy added a consumer when the incoming RPS grew and that a real Kafka rebalance followed.

8. References

1. R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
2. Apache Kafka Documentation: consumer groups and rebalance. https://kafka.apache.org/documentation/
3. KIP-429: Kafka Consumer Incremental Rebalance Protocol. https://cwiki.apache.org/confluence/display/KAFKA/KIP-429
4. KEDA documentation: Apache Kafka scaler. https://keda.sh/docs/latest/scalers/apache-kafka/
5. J. D. C. Little. A proof for the queuing formula L = λW. *Operations Research*, 9(3), 1961.
6. CSCI3235 Midterm I Take Home Project description, KBTU, 2026.

Appendix: Learning record

1. Course work before the midterm week

Activity	What I did	Evidence
CSCI3235 practice	Completed and defended the RL practice assignment in class	Defended in class
Coursera module defence	Defended my Coursera RL module progress in class	Defended in class

2. Coursera: Reinforcement Learning Specialization

Course 1, *Fundamentals of Reinforcement Learning* (University of Alberta). Completed modules:

Module	Main ideas
An Introduction to Sequential Decision-Making	k-armed bandits, action values, incremental update, ε-greedy exploration
Markov Decision Processes	agent–environment loop, states, actions, rewards, episodic vs continuing tasks, return and discounting
Value Functions & Bellman Equations	state and action values, policies, Bellman equations, optimal values

3. Independent study and project work during the midterm week

- Studied tabular Q-learning (Sutton & Barto, Section 6.5) and applied it in the LagPilot project.
- Studied Apache Kafka consumer groups, consumer lag, rebalancing and the KEDA Kafka scaler, and Little's Law from my *Backend for Highloaded Environment* course.
- Ran the live Kafka demo myself (FastAPI producer, load generator, Q-table agent) and read the agent's decisions in its log.
- Evidence: project repository with commit history; `results/` (figures and CSVs); `demo/sample_run.log`.

4. Reflection

An idea that became clearer. In the bandit module I learned the incremental rule: new estimate = old estimate + step size × (target − old estimate). In this project I saw that Q-learning uses exactly the same rule. The only difference is the target: instead of the reward alone, it is the reward plus γ times the best Q-value of the next state. The MDP and Bellman modules explained why this extra term is needed: an action changes the next state, so its value depends on the future. In my problem this is very concrete. Adding a Kafka consumer starts a rebalance, which stops processing and makes the lag grow in the next steps, so the cost of the action appears only later. Reading the agent's decisions in the live demo log made this loop of state, action, reward and next state feel real.

An idea that is still difficult. I still find it hard to judge when a state is "Markov enough" and how to choose the discretization. My state hides the exact lag and the time of day, and one of my ten training seeds learned a bad policy. I do not yet fully understand why tabular Q-learning becomes unstable in rarely visited states, or how to fix it reliably with a different step size or more training.